

Événements personnalisés

En JavaScript, il est possible d'écouter un événement autre que ceux fournis nativement par le langage. Pour cela, on peut utiliser la méthode `CustomEvent` pour créer un événement personnalisé, puis la méthode `dispatchEvent` pour déclencher cet événement.

```
// Création d'un événement personnalisé
const monEvenement = new CustomEvent("monEvenement", {
  detail: {
    message: "Ceci est un événement personnalisé",
  },
});

// Écoute de l'événement dans n'importe quel élément du DOM
document.addEventListener("monEvenement", (e) => {
  console.log(e.detail.message);
});

// Déclenchement de l'événement
document.dispatchEvent(monEvenement);
```

Syntaxe du CustomEvent

On commence par créer un nouvel événement personnalisé en utilisant le constructeur `CustomEvent`. Ce constructeur prend deux arguments : le nom de l'événement et un objet d'options. L'objet d'options peut contenir une propriété `detail` qui transporte des données supplémentaires avec l'événement.

Vous pouvez donner n'importe quel nom à votre événement personnalisé, mais il est recommandé d'utiliser un nom unique pour éviter les conflits avec d'autres événements.

```
const monEvenement = new CustomEvent("monEvenement", {
  detail: {
    message: "Ceci est un événement personnalisé",
    id: 3,
    tableauProduits: ["Produit 1", "Produit 2"],
  },
});
```

Ensuite, on peut écouter cet événement sur n'importe quel élément du DOM en utilisant `addEventListener`.

```
document.addEventListener("monEvenement", (e) => {
  console.log(e.detail.message);
});
```

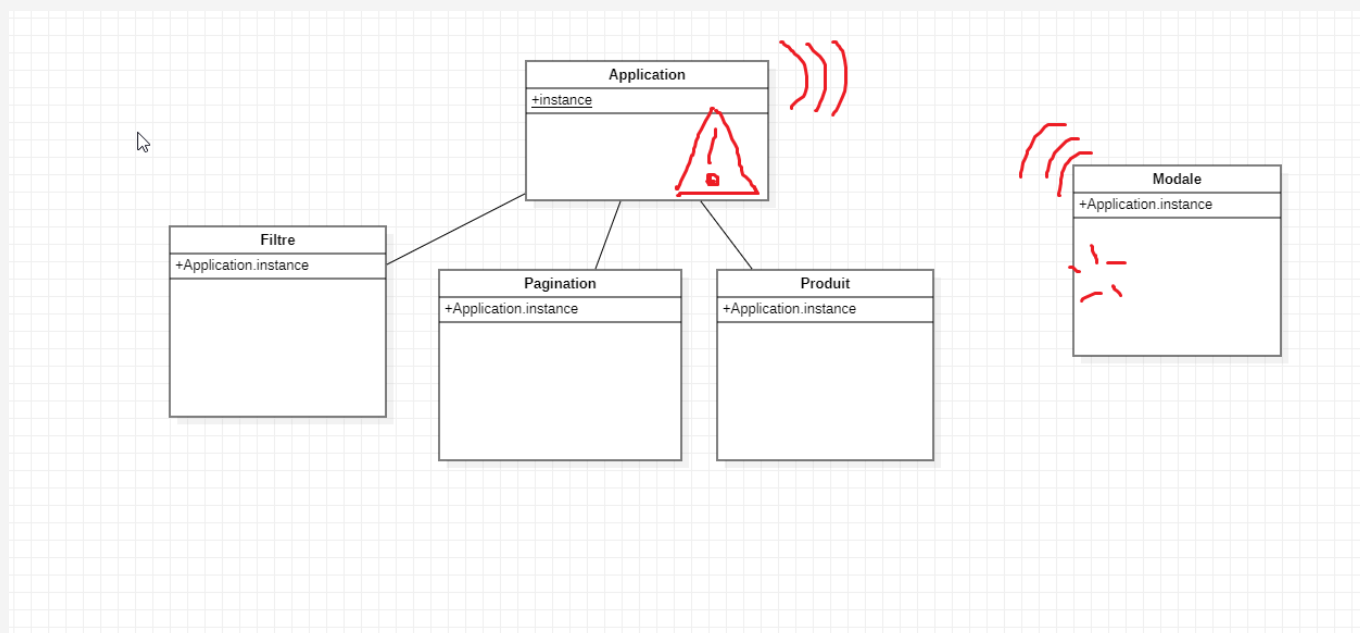
Enfin, pour déclencher l'événement, on utilise `dispatchEvent`. Le dispatch doit obligatoirement être effectué sur l'élément qui écoute l'événement. C'est fréquent d'utiliser le document comme élément du DOM avec les événements personnalisés.

```
document.dispatchEvent(monEvenement);
```

Patron de conception Observer

Grâce aux événements personnalisés, nous pourrions simplifier le problème soulevé au cours passé. C'est-à-dire que nous pouvons maintenant communiquer entre les classes de manière plus efficace en utilisant des événements au lieu de lier directement les instances de classes entre elles.

Par exemple, une classe de boîte modale, n'a pas besoin d'être liée directement à la classe Application. Au lieu de cela, elle peut écouter un événement personnalisé lorsqu'une action se produit (comme s'afficher lors d'une erreur), et la classe Application peut déclencher cet événement lorsqu'elle le souhaite. Encore mieux, plusieurs classes différentes peuvent attendre le même événement pour se mettre à jour.



Exemple

```
class Application {
  constructor() {
    this.#produits = [];
    this.#conteneurHTML = document.querySelector("[data-application]");
    // Initialisation de l'application
    if (this.#conteneurHTML == null) {
      this.declencherEvenement();
    }
  }

  declencherEvenement() {
    // Déclenchement de l'événement personnalisé
    const monEvenement = new CustomEvent("afficherErreur", {
```

```

        detail: {
            message: "Ceci est un événement déclenché par l'application",
            id: 3,
        },
    });
    document.dispatchEvent(monEvenement);
}
}

class Modal {
    constructor() {
        this.#conteneurHTML = document.querySelector("[data-modal]");
        //On attend les événements personnalisés et on réagit en conséquence si
        besoin
        document.addEventListener("afficherErreur",
        this.afficherErreur.bind(this));
    }

    afficherErreur(evenement) {
        const detailEvenement = evenement.detail;
        console.error(detailEvenement.message, detailEvenement.id);
    }
}

```